

● NAVEGAÇÃO

<code>pwd</code>	mostra o diretório atual
<code>ls</code>	lista arquivos
<code>ls -la</code>	lista com detalhes e ocultos
<code>cd pasta/</code>	entra em um diretório
<code>cd ..</code>	volta um nível
<code>cd ~</code>	vai para o diretório home
<code>clear</code>	limpa a tela do terminal

● ARQUIVOS & PASTAS

<code>mkdir nome</code>	cria um diretório
<code>mkdir -p a/b/c</code>	cria estrutura aninhada
<code>touch arq</code>	cria arquivo vazio
<code>echo "txt" > arq</code>	escreve texto em arquivo
<code>cat arq</code>	exibe conteúdo do arquivo
<code>cp origem dest</code>	copia arquivo
<code>cp -r dir/ dest/</code>	copia diretório inteiro
<code>mv origem dest</code>	move ou renomeia
<code>rm arq</code>	remove arquivo
<code>rm -r pasta/</code>	remove pasta e conteúdo

● ATALHOS ÚTEIS

<code>Tab</code>	autocompleta comandos e nomes
<code>↑ ↓</code>	navega no histórico de comandos
<code>Ctrl+C</code>	cancela o comando atual
<code>Ctrl+L</code>	limpa a tela (igual a clear)
<code>man cmd</code>	abre o manual do comando
<code>history</code>	exibe histórico de comandos

● .GITIGNORE — O QUE IGNORAR

<code># comentário</code>	
<code>node_modules/</code>	← ignora pasta inteira
<code>*.log</code>	← ignora todos .log
<code>.env</code>	← nunca commitar senhas!
<code>dist/</code>	← arquivos gerados

● CONFIGURAÇÃO INICIAL

<code>git config --global user.name "Nome"</code>	define seu nome
<code>git config --global user.email "e@mail"</code>	define seu e-mail
<code>git config --list</code>	vê as configurações atuais

● REPOSITÓRIO LOCAL

<code>git init</code>	inicializa um repositório
<code>git status</code>	mostra estado dos arquivos
<code>git add arq</code>	adiciona arquivo ao stage
<code>git add .</code>	adiciona tudo ao stage
<code>git commit -m "msg"</code>	cria um commit
<code>git log</code>	exibe histórico de commits
<code>git log --oneline</code>	histórico resumido
<code>git diff</code>	mostra mudanças não staged
<code>git show hash</code>	detalhes de um commit

● BRANCHES

<code>git branch</code>	lista branches
<code>git branch nome</code>	cria uma nova branch
<code>git checkout nome</code>	troca para a branch
<code>git checkout -b nome</code>	cria e já troca de branch
<code>git merge nome</code>	faz merge da branch na atual
<code>git branch -d nome</code>	deleta a branch

● ERROS COMUNS & SOLUÇÕES

<code>not a git repository</code>	rode <code>git init</code> antes
<code>nothing to commit</code>	faça <code>git add .</code> antes do commit
<code>rejected (non-fast-forward)</code>	rode <code>git pull</code> antes do push
<code>detached HEAD</code>	rode <code>git checkout main</code>

● GITHUB / REMOTO

<code>git remote add origin URL</code>	conecta ao repositório remoto
<code>git remote -v</code>	lista remotos configurados
<code>git push -u origin main</code>	envia commits ao GitHub
<code>git push</code>	envia (após -u configurado)
<code>git pull</code>	atualiza do remoto
<code>git clone URL</code>	clona repositório remoto
<code>git fetch</code>	baixa mudanças sem aplicar

● FLUXO GIT — CICLO DE VIDA



EXEMPLO: FLUXO COMPLETO

```
$ git init
$ git add .
$ git commit -m "feat: página inicial"
$ git remote add origin <URL>
$ git push -u origin main
```

● BOAS PRÁTICAS DE COMMIT

- Use verbos no imperativo: **adiciona, corrige, remove**
- Mantenha a mensagem curta e descritiva — máx. 72 chars
- Commit pequeno = mais fácil de revisar e desfazer
- Nunca commite arquivos desnecessários — use **.gitignore**
- Prefixo útil: **feat:** nova função · **fix:** correção · **docs:** documentação

● PRÓXIMOS PASSOS

- git-scm.com/book → Pro Git, livro gratuito e completo
- learngitbranching.js.org → visualizar branches interativamente
- GitHub Student Developer Pack — ferramentas gratuitas para estudantes